

Refactoring: The `_check_events()` and `_update_screen()` Methods

In large projects, you'll often refactor code you've written before adding more code. Refactoring simplifies the structure of the code you've already written, making it easier to build on. In this section, we'll break the `run_game()` method, which is getting lengthy, into two helper methods. A *helper method* does work inside a class but isn't meant to be used by code outside the class. In Python, a single leading underscore indicates a helper method.

The `_check_events()` Method

We'll move the code that manages events to a separate method called `_check_events()`. This will simplify `run_game()` and isolate the event management loop. Isolating the event loop allows you to manage events separately from other aspects of the game, such as updating the screen.

Here's the `AlienInvasion` class with the new `_check_events()` method, which only affects the code in `run_game()`:

```
alien
_invasion.py    def run_game(self):
                  """Start the main loop for the game."""
                  while True:
❶                 self._check_events()

                  # Redraw the screen during each pass through the loop.
                  --snip--

❷    def _check_events(self):
        """Respond to keypresses and mouse events."""
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                sys.exit()
```

We make a new `_check_events()` method ❷ and move the lines that check whether the player has clicked to close the window into this new method.

To call a method from within a class, use dot notation with the variable `self` and the name of the method ❶. We call the method from inside the while loop in `run_game()`.

The `_update_screen()` Method

To further simplify `run_game()`, we'll move the code for updating the screen to a separate method called `_update_screen()`:

```
alien_invasion.py    def run_game(self):
                      """Start the main loop for the game."""
                      while True:
                          self._check_events()
                          self._update_screen()
                          self.clock.tick(60)

                      def _check_events(self):
                          --snip--
```
